

Daily Paper Review: The Mirage of Optimizing Training Policies

Monotonic Inference Policies as the Real Objective for LLM Reinforcement Learning

Internbot — Automated Research Summary

July 7, 2026

1 Paper Summary

1.1 Problem and Motivation

Liang et al. [1] identify a fundamental objective misalignment in modern LLM reinforcement learning pipelines: the policy being *optimized* during training is not the same policy that is *deployed* for inference. In production RL systems, rollout generation is handled by an inference engine (e.g., vLLM with FP8 quantization) while gradient computation runs in a separate training engine (e.g., Megatron with BF16/FP32 precision). Even when parameters are synchronized between the two engines, differences in numerical precision, kernel implementations, and serving backends cause the **training policy** π and the **inference policy** μ to assign different probabilities to the same token sequences.

The Mirage. The paper’s central thesis is that improving the training policy does *not* guarantee improving the inference policy:

$$J(\pi_{k+1}) - J(\pi_k) \geq 0 \not\Rightarrow J(\mu_{k+1}) - J(\mu_k) \geq 0.$$

Standard RL algorithms (GRPO, PPO) optimize $J(\pi)$ under the implicit assumption that gains transfer to μ . The authors call this assumption a “mirage”—apparent progress on the training side that may not materialize, or may even *reverse*, when the model is deployed via the inference engine. This reframes training-inference mismatch from a systems-engineering nuisance into a **fundamental optimization objective problem**.

Two Sources of Mismatch in GRPO. Under mismatch, GRPO suffers from: (1) **Ratio-level mismatch**: the importance sampling ratio $r_i(\theta) = \pi_\theta(y|x)/\pi_k(y|x)$ is computed entirely on the training side, but samples come from μ_k ; and (2) **Advantage-level bias**: GRPO computes group-relative advantages from responses sampled by μ_k , so the advantage statistic $\hat{A}^{\mu_k}$ is not aligned with the training-policy advantage A^{π_k} .

1.2 Method

The paper proposes **Monotonic Inference Policy Update (MIPU)**, a two-step framework grounded in a new optimization principle called **Monotonic Inference Policy Improvement (MIPI)**.

The MIPI Decomposition. The inference-policy improvement decomposes into three terms:

$$J(\mu_{k+1}) - J(\mu_k) = \underbrace{[J(\mu_{k+1}) - J(\pi_{k+1})]}_{\text{Term 1: post-update gap}} + \underbrace{[J(\pi_{k+1}) - J(\pi_k)]}_{\text{Term 2: training update}} + \underbrace{[J(\pi_k) - J(\mu_k)]}_{\text{Term 3: pre-update gap}}. \quad (1)$$

Standard LLM RL only optimizes Term 2. MIPU addresses all three terms: Step 1 handles Terms 2+3 jointly, and Step 2 validates Term 1.

Step 1: Sampler-Referenced Policy Update. Instead of computing the importance ratio relative to the old training policy π_k , Step 1 references the inference (sampler) policy μ_k . The full trainer-to-sampler ratio factorizes as:

$$\rho_i(\theta) = \frac{\pi_\theta(y|x)}{\mu_k(y|x)} = \underbrace{\frac{\pi_k(y|x)}{\mu_k(y|x)}}_{w_i^k: \text{mismatch correction}} \cdot \underbrace{\frac{\pi_\theta(y|x)}{\pi_k(y|x)}}_{r_i(\theta): \text{update ratio}}.$$

The paper evaluates three implementations: (a) PPO-IS, which clips the full ratio ρ_i directly but suffers from over-active clipping; (b) Vanilla-IS, which uses unbounded w_i^k but has high gradient variance; and (c) **Truncated IS (TIS)**, which truncates w_i^k at $w_{\max} = 2$ while applying PPO clipping only to $r_i(\theta)$. TIS provides the best stability-performance trade-off and is used in the final method.

The Step 1 surrogate objective is:

$$J_{S1}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\} \sim \mu_k} \left[\frac{1}{G} \sum_{i=1}^G \bar{w}_i^k \cdot \min(r_i(\theta) \hat{A}_i^{\mu_k}, \text{clip}(r_i(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_i^{\mu_k}) \right],$$

where $\bar{w}_i^k = \min(w_i^k, w_{\max})$ is the truncated mismatch weight, computed at the **token level**. A dual-clipped loss adds a lower bound $(1+2\epsilon)\hat{A}$ for negative advantages to prevent ratio collapse.

Step 2: Inference-Gap-Aware Update Acceptance. After Step 1 produces candidate π_{k+1} and it is synchronized to the inference engine as μ_{k+1} , Step 2 estimates the post-update inference gap (Term 1) using a validation-based proxy:

$$\hat{T}_{\text{post}} = -\mathbb{E}_{x \sim \mathcal{D}_{\text{val}}, y_i \sim \mu_{k+1}} \left[\rho_i \cdot \hat{A}_i^{\mu_{k+1}} \right],$$

where ρ_i is a **length-normalized sequence ratio**:

$$\rho_i = \exp \left(\frac{1}{T_i} \sum_{t=1}^{T_i} \log \frac{\pi_{k+1}(y_{i,t} \mid x, y_{i,<t})}{\mu_{k+1}(y_{i,t} \mid x, y_{i,<t})} \right).$$

Length normalization avoids the high variance of full sequence-level importance sampling. The acceptance criterion is:

$$\hat{T}_{\text{post}} \geq -c \implies \text{Accept}; \quad \hat{T}_{\text{post}} < -c \implies \text{Rollback},$$

where $c \geq 0$ is a tolerance parameter, dynamically annealed from c_{start} to c_{end} over the first 100 steps (lenient early, tighter later). When rejected, both trainer state (parameters + optimizer) and inference engine are rolled back to the previous checkpoint.

1.3 Results

Setup. Qwen3-1.7B and Qwen3-4B trained with GRPO on math reasoning tasks (DAPO-Math-17k and DeepMath-103K respectively). All rollouts use FP8-quantized vLLM to amplify training-inference mismatch. Evaluation: Pass@1 on MATH-500, AIME24 (avg@16), AMC23 (avg@16), Minerva, OlympiadBench. $8 \times$ H100 GPUs.

Main Results. On Qwen3-4B, MIPU achieves the best average across all benchmarks:

- MIPU: **66.71%** average (MATH 91.15, AIME24 43.56, Olympiad 67.86, Minerva 45.96, AMC23 85.00).
- Best baseline (LR-decay): 65.66%. Standard GRPO: 64.42%. MIS: 63.42%.

On Qwen3-1.7B, MIPU achieves **53.97%** average vs. LR-decay 52.23% and GRPO 50.86%. Crucially, MIPU is the **only method that remains stable**—all baselines suffer from training collapse or sharp performance degradation during continued training.

Ablation: Both Steps Are Necessary. On Qwen3-4B:

- Step 1 alone: 65.36% (improves candidate quality but cannot filter bad synchronized updates).
- Step 2 alone: 62.81% (can prevent collapse by rejecting bad candidates, but cannot improve the underlying update quality—actually *worse* than baseline).
- Both steps: **66.71%** (complementary: Step 1 generates better candidates, Step 2 validates them).

Signal vs. Random Rollback. Random rollback at 70% rate (matching Step 2’s rejection frequency) still collapsed, proving that Step 2’s benefit comes from the $\hat{T}_{\text{post}}$ signal, not from merely reducing update frequency.

Step 1 Variant Analysis. Among the three Step 1 implementations: PPO-IS reduces measured KL but fails to sustain learning due to over-active clipping; Vanilla-IS shows clearer upward trends but has gradient-norm spikes from unbounded weights; TIS provides the best trade-off.

1.4 Limitations

Limited model scale: Experiments are restricted to 1.7B and 4B parameters. Whether the same mismatch patterns and MIPU benefits appear at 70B+ scale is unknown.

No formal guarantee: The authors explicitly state that MIPU “does not provide a formal monotonic-improvement guarantee.” The method reduces the risk of harmful updates but cannot provably ensure $J(\mu_{k+1}) \geq J(\mu_k)$ at every step.

Computational overhead: Step 2 requires additional validation rollouts and checkpoint/rollback infrastructure at every step. The wall-clock overhead is not quantified.

Single mismatch source: All experiments use FP8 quantization as the mismatch source. Other mismatch sources (different decoding strategies, tensor parallelism, attention implementations) are untested.

Single task domain: All experiments use mathematical reasoning with verifiable rewards. Transfer to tasks with learned reward models (RLHF for alignment) is untested, where the reward signal itself may be noisy.

High rollback rate: Step 2 alone rejects $\sim 70\%$ of updates in the first 500 steps, meaning most training computation is wasted. Even the full method likely discards a significant fraction of gradient updates.

1.5 Relevance to Our Research

This paper provides the most principled treatment of training-inference mismatch in LLM RL to date, directly advancing our T2 (RL/Reward Modeling) research thread. Our extensive corpus of RL reviews has studied various aspects of policy optimization: BandPO [2] addressed trust region vs. ratio clipping, DelTA [3] tackled token-level credit assignment, DVAO [4] handled multi-reward variance, and ReNIO [5] reweighted negative trajectories. All of these methods implicitly assume that the policy being optimized is the policy being deployed. The MIPI decomposition reveals this assumption is violated in every real production system, suggesting that *all* of our previously reviewed RL methods could benefit from MIPU-style inference-gap awareness.

The connection to our distillation research (T1) is particularly striking. Our on-policy distillation (OPD) reviews—DOPD [6], Filter-Then-Reweight [7], ReNIO [5], Early Stopping Rollout [8]—all use the student policy for rollout generation while computing losses relative to both student and teacher. The MIPI decomposition applies directly: if the student’s rollout engine uses quantized weights (common in practice), the “teacher” signal is computed against a different policy than the one generating trajectories. MIPU’s sampler-referenced correction (Step 1) could be adapted to OPD by treating the quantized student rollout policy as μ and the full-precision student as π .

The Verification Horizon [9] from our recent reviews showed that reward design for coding agents faces a scalability-faithfulness-robustness trilemma. The MIPI framework reveals an orthogonal dimension: even with a perfect reward function, the RL update may fail to improve the deployed policy due to training-inference mismatch. This suggests that the “verification horizon” is even harder to reach than previously thought—not only must the reward be faithful, but the optimization must also survive the training-to-inference transfer.

Step 2’s rollback mechanism connects to our continual learning thread (T3). Our EvoArena [10] and Geometry Conflict [11] reviews studied how to detect and prevent catastrophic forgetting during post-training. MIPU’s inference-gap-aware acceptance criterion is conceptually similar: it detects when an update *appears* beneficial on the training side but is actually harmful on the deployment side, and rolls back. The difference is that MIPU addresses mismatch-induced degradation rather than forgetting, but the checkpoint-and-validate paradigm is shared.

2 Follow-up Research Directions

2.1 Direction 1: Inference-Gap-Aware On-Policy Distillation

Gap. Our on-policy distillation (OPD) pipeline generates student rollouts using an inference engine (often with quantized weights for efficiency) while computing distillation losses in a training engine at full precision. The MIPI decomposition applies directly to this setting: the student’s training policy π_{student} and inference policy μ_{student} differ, so optimizing the training-side distillation loss may not improve the deployed student. None of our OPD reviews (DOPD [6], Filter-Then-Reweight [7], ReNIO [5], Early Stopping Rollout [8]) account for this mismatch.

Approach. Adapt MIPU to the OPD setting with three modifications:

1. **Sampler-referenced distillation loss:** Replace the standard OPD ratio $\pi_{\text{student}}^{\text{train}}(y|x)/\pi_{\text{student}}^{\text{old}}(y|x)$ with the mismatch-corrected ratio $\bar{w}_i \cdot r_i(\theta)$, where $\bar{w}_i$ corrects for the gap between the quantized rollout student μ_{student} and the full-precision training student π_{student} .
2. **Teacher-aware acceptance:** Extend Step 2’s acceptance criterion to include teacher agreement. After synchronizing the updated student to the inference engine, verify not

only that $\hat{T}_{\text{post}} \geq -c$ but also that the teacher-student KL divergence on the validation set has decreased. This prevents accepting updates where the mismatch correction inadvertently moves the student *away* from the teacher.

3. **Privilege-stratified mismatch correction:** Drawing on DOPD’s privilege advantage gap concept, apply different mismatch correction strengths to different token regimes. High-privilege tokens (where the teacher has a large capability advantage) should receive stronger mismatch correction because errors on these tokens are more consequential.

Feasibility. The mismatch correction weights w_i are already computable from the training and inference log-probabilities (both available in standard OPD pipelines). The teacher-aware acceptance adds one forward pass through the teacher per validation step (affordable since the teacher is already loaded for distillation). Validation: compare standard DOPD vs. MIPU-augmented DOPD on Qwen3-4B→Qwen3-1.7B distillation with FP8 student rollouts. Expected outcome: MIPU-augmented DOPD improves the distilled student by 2–4% on math benchmarks while eliminating the training instability that FP8 rollouts introduce.

2.2 Direction 2: Continuous Inference Gap Estimation via Lightweight Proxy Networks

Gap. MIPU’s Step 2 requires generating full validation rollouts from μ_{k+1} at every training step, introducing significant computational overhead. The paper does not quantify this overhead, but generating G responses per validation prompt at every step effectively doubles the rollout cost. For large-scale training (70B+ models), this overhead may be prohibitive. Can we estimate the inference gap *without* additional rollouts?

Approach. Train a lightweight **inference gap predictor** that estimates $\hat{T}_{\text{post}}$ from cheaply observable signals:

1. **Feature extraction:** At each training step, compute a feature vector from: (a) the gradient norm of the current update, (b) the mean and variance of token-level mismatch weights w_i^k from Step 1, (c) the KL divergence between π_{k+1} and π_k on the training batch, and (d) the change in training loss. These are all byproducts of the training step—no additional forward passes needed.
2. **Proxy network:** A small MLP (2 layers, 64 hidden units) that maps the feature vector to a predicted $\hat{T}_{\text{post}}$. Trained online using the actual $\hat{T}_{\text{post}}$ values from Step 2 rollouts as labels during the first N steps (warm-up phase).
3. **Phased deployment:** During the warm-up phase (first N steps), run full Step 2 rollouts and train the proxy. After warm-up, use the proxy for acceptance decisions, falling back to full Step 2 rollouts every M steps to calibrate and retrain the proxy.

This amortizes the validation cost: instead of rolling out at every step, the proxy provides near-instant acceptance decisions using only training-side statistics. The intermittent full rollouts prevent proxy drift.

Feasibility. The feature extraction adds negligible computation (gradient norms and mismatch weights are already computed). The MLP proxy has <10K parameters and trains in milliseconds. The key question is whether the proxy generalizes across training steps—if the relationship between training-side statistics and inference-side quality is stable (which the MIPI decomposition suggests it should be, since Term 1 depends on the mismatch structure which changes slowly), the proxy should maintain good accuracy. Validation: compare full MIPU vs.

proxy-augmented MIPU on Qwen3-4B, measuring both final performance and wall-clock time. Expected outcome: proxy-augmented MIPU achieves 95%+ of full MIPU’s performance gains while reducing validation overhead by 80%.

2.3 Direction 3: Mismatch-Aware Reward Model Training

Gap. The MIPI framework focuses on the policy optimization loop, but training-inference mismatch also affects **reward models**. In RLHF pipelines, the reward model is trained to score responses from the policy, but the policy that generated training data (the inference policy μ) differs from the policy against which the reward model’s gradients are computed (the training policy π). Our reward modeling reviews—Beyond Length Scaling [12], Beyond Scalar Rewards [13], Verification Horizon [9]—all assume the reward model evaluates the “true” policy. If the policy shift from μ to π changes which responses the reward model sees, the reward signal may be systematically biased.

Approach. Design a **mismatch-robust reward model** that accounts for policy distribution shift:

1. **Distribution-calibrated reward:** Instead of training $R(x, y)$ to match human preferences on samples from a fixed reference policy, train on samples from *both* μ and π , weighting each sample by the mismatch ratio $w = \pi(y|x)/\mu(y|x)$. This ensures the reward model is calibrated for the actual training-side distribution.
2. **Mismatch-aware preference pairs:** When constructing preference pairs (y^+, y^-) , ensure that both responses are evaluated under both policies. If a response is preferred under μ but not under π (or vice versa), flag it as a **mismatch-sensitive pair** and upweight it during reward training to ensure the reward model resolves the ambiguity.
3. **Reward consistency regularization:** Add a regularization term $\lambda \cdot |R(x, y; \mu) - R(x, y; \pi)|$ that penalizes the reward model for assigning different scores to the same response when evaluated under different policy contexts. This encourages the reward to be robust to the training-inference distribution shift.

Feasibility. Mismatch ratios are already computed in MIPU’s Step 1. Preference pair construction from both policies adds one additional forward pass per pair (through the alternative policy). The consistency regularization is a standard penalty term. Validation: compare standard reward model vs. mismatch-aware reward model in an RLHF pipeline with FP8 rollouts on a non-math task (e.g., AlpacaEval for instruction following). Expected outcome: mismatch-aware reward model reduces reward hacking by 20–30% and improves final policy quality by 1–2 points on AlpacaEval, by providing a reward signal that accurately reflects the deployed policy’s behavior.

3 Conclusion

The MIPI decomposition reveals that modern LLM RL has been optimizing a mirage: training-side policy improvement does not guarantee inference-side improvement when the two engines produce different probability distributions. MIPU’s two-step solution—sampler-referenced policy update (correcting the optimization direction) and inference-gap-aware acceptance (validating the synchronized result)—is both theoretically principled and practically effective, achieving the best average performance across all benchmarks while being the only method to maintain training stability. The ablation revealing that Step 2 alone actually *hurts* performance is particularly illuminating: blindly rejecting updates based on an inference gap signal without first

improving candidate quality leads to stagnation, demonstrating that the two steps are genuinely complementary rather than redundant. For our research program, the MIPI framework has immediate implications for every RL and distillation method we have reviewed—any method that uses separate engines for rollout and training is implicitly ignoring Terms 1 and 3 of the decomposition. The three follow-up directions—inference-gap-aware OPD (bridging T1 and T2), proxy-based gap estimation (making MIPU scalable to 70B+ models), and mismatch-aware reward modeling (protecting the reward signal from distribution shift)—collectively extend the “mirage” insight from policy optimization to the entire LLM post-training stack.

References

- [1] Liang, J., Tang, H., Ma, Y., et al. “The Mirage of Optimizing Training Policies: Monotonic Inference Policies as the Real Objective for LLM Reinforcement Learning.” *arXiv preprint arXiv:2606.29526*, 2026.
- [2] BandPO Authors. “BandPO: Bridging Trust Regions and Ratio Clipping via Probability-Aware Bounds for LLM Reinforcement Learning.” *arXiv preprint arXiv:2603.04918*, 2026.
- [3] DelTA Authors. “DelTA: Discriminative Token Credit Assignment for Reinforcement Learning from Verifiable Rewards.” *arXiv preprint arXiv:2605.21467*, 2026.
- [4] DVAO Authors. “DVAO: Dynamic Variance-adaptive Advantage Optimization for Multi-reward Reinforcement Learning.” *arXiv preprint arXiv:2605.25604*, 2026.
- [5] Lin, C., et al. “ReNIO: Reweighting Negative Trajectory Importance for LLM On-Policy Distillation.” *arXiv preprint arXiv:2606.23104*, 2026.
- [6] Yu, X., et al. “DOPD: Dual On-Policy Distillation.” *arXiv preprint arXiv:2606.30626*, 2026.
- [7] Filter-Then-Reweight Authors. “Filter, Then Reweight: Rethinking Optimization Granularity in On-Policy Distillation.” *arXiv preprint arXiv:2606.02684*, 2026.
- [8] ESR Authors. “Less is More: Early Stopping Rollout for On-Policy Distillation.” *arXiv preprint arXiv:2605.27028*, 2026.
- [9] Verification Horizon Authors. “The Verification Horizon: No Silver Bullet for Coding Agent Rewards.” *arXiv preprint arXiv:2606.26300*, 2026.
- [10] EvoArena Authors. “EvoArena: Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments.” *arXiv preprint arXiv:2606.13681*, 2026.
- [11] Geometry Conflict Authors. “Geometry Conflict: Explaining and Controlling Forgetting in LLM Continual Post-Training.” *arXiv preprint arXiv:2605.09608*, 2026.
- [12] BLS Authors. “Beyond Length Scaling: Synergizing Breadth and Depth for Generative Reward Models.” *arXiv preprint arXiv:2603.01571*, 2026.
- [13] BSR Authors. “Beyond Scalar Rewards by Internalizing Reasoning into Score Distributions.” *arXiv preprint arXiv:2606.09076*, 2026.